



Workflow

Historique des versions

Version	Par qui ?	Quand ?	Quoi ?	Pourquoi ?
1.0	Maéva De Campos	05/02/24	Tout	Création du document
2.0	Jalil Jaajoui	07/05/24	Application du Template et clarifications	Clarifier Homogénéiser le documents selon Charte Graphique



Sommaire

Introduction.....	5
1) Workflow.....	6
1.1) Framework inspiré de Scrum.....	7
1.2) Rôles.....	8
1.3) Artefacts.....	8
1.4) Événements.....	8
1.5) L'outil Jira.....	9
2) Git Flow.....	9
2.1) Distinction entre branche Main et Dev.....	9
2.2) Création de branche.....	9
2.2.3) Convention de nommage des branches.....	10
2.3) Phase de test avant de merge.....	10
2.3.1) Respect des spécifications.....	10
2.3.2) Passer les tests, vérification de la feature.....	10
2.3.3) Comment merge?.....	10
2.3.4) Open a Pull Request.....	12
2.3.5) Code Review.....	12
3) Description Workflow Jira.....	13



Introduction

Utilisation Agile:

Ensemble de pratiques visant à gagner en transparence, efficacité en promouvant l'auto-gestion de l'équipe basée sur la confiance et la maturité de cette dernière à apprendre de ses erreurs afin d'ajuster l'organisation.

Organisation de l'épique et définition des rôles:

- Scrum Master
- Product Owner (rédaction des US)
- Developers

Chaque membre devra s'assurer de s'éduquer à la méthode agile et accepter d'adhérer à ses valeurs et principes:

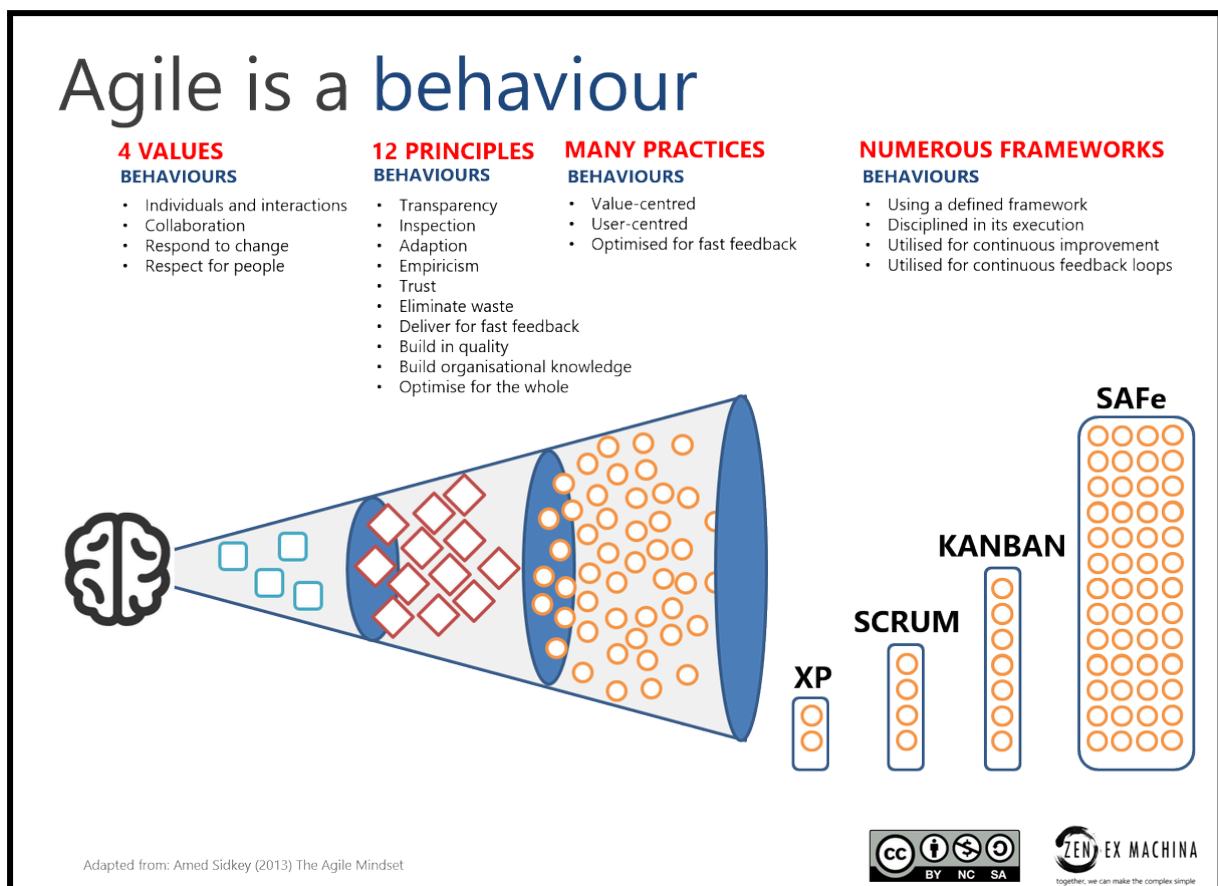
- Daily Scrum
- Rétrospectives Scrum
- Backlog produit
- Backlog Sprint



1) Workflow

- La **méthode en cascade** (waterfall) ne conviendra pas au projet pour le simple fait que les parties sont dépendantes les unes des autres. Un fonctionnement qui, par la composition de l'équipe et la répartition du temps de travail (estimé à une demi-journée/semaine officiellement) causera de gros ralentissement par définition.
- Le **cycle en V** ne correspond pas aux besoins du projet qui est suivi par un FU mensuel on nous devrions être capable de montrer l'avancement concret du projet.
- Seule l'**agilité** nous permettra de travailler en simultané sur divers sujets du projet en le délimitant par des users stories estimées en jour homme.

Pour ces raisons, nous avons décidé d'évoluer dans un **contexte agile**.



L'agilité étant avant tout un état d'esprit avec **4 valeurs et 12 principes** pouvant se manifester au travers de diverses pratiques.

Nous avons choisi le **framework Scrum** qui est une manière parmi tant d'autres d'appliquer l'agilité.

1.1) Framework inspiré de Scrum

- Il convient aux besoins de **flexibilité** du projet
- Il permettra de pouvoir **travailler sur des blocs indépendant** du projet
- Basé sur la confiance, il promeut l'**autonomie** des parties prenantes
- Garanti **un livrable à chaque étape** défini par les membres de l'équipe
- Il permettra de pouvoir **réajuster le projet** rapidement pour coller au mieux aux attentes et objectifs fixés au début du projet
- Il laisse la possibilité de **réajuster les attentes du projet** en cours de développement suite à des montées de compétences sur des technologies et une meilleure visibilité des possibilités et des limitations.
- L'équipe est volontaire et force de proposition afin d'ajuster la forme que devront prendre les divers événements qui caractérise le framework Scrum (daily, rétro, affinage du backlog...)
- Rien n'est figé, les valeurs et principes de l'agilité peuvent se manifester de manières diverses, à l'équipe de trouver la méthode qui lui convient
- Garantir le bien-être de chacun dans l'équipe, s'assurer que toutes les voix sont entendu

1.2) Rôles

- **Product Owner** : Définit les User Story au début du projet et peut les réorganiser tout le long.
- **SCRUM Master** : Organise et dirige les événements SCRUMS tels que les Sprint Planning.
- **Membre de la team** : Crée des tickets de tâches lorsqu'il s'attribue une User Story. Met à jour ses tickets tout au long du sprint.

1.3) Artefacts

- **Product Backlog** : Liste ordonnée des éléments nécessaires pour améliorer le produit.
- **Sprint Backlog** : Liste détaillée des tâches pour accomplir l'objectif de Sprint.
- **Sprint Table** : Tableau contenant l'avancement des tâches et des User Story du sprint.

1.4) Événements

- **Sprint** : Cadre de travail d'une durée fixe 1 mois
- **Sprint Planning** : Planifie le travail à accomplir pendant le Sprint (Au début de chaque sprint)
- **Weekly Scrum** : Réunion hebdomadaire pour inspecter la progression. idéalement Lundi à 15H.
- **Sprint Review** : Démonstration des résultats du Sprint avec les membres de l'équipe. (Avant le follow up)
- **Sprint Retrospective** : Réflexion sur le dernier Sprint pour identifier les améliorations. (Avant le follow up)

1.5) L'outil Jira

Pour mettre en place le framework inspiré de Scrum, nous avons choisis l'outil Jira qui permettra une gestion optimale des tickets.

2) Git Flow

2.1) Distinction entre branche Main et Dev

- La branche principale contiendra des **commits abrégés**
- La branche de développement contiendra **tout l'historique des commits**
- Il est important de **ne travailler que sur la branche develop**
- La branche **master doit toujours être prête à être déployée.**

2.2) Création de branche

Vous ne devez jamais commit directement dans dev, mais plutôt créer des branches à partir de dev. Avant de créer sa branche il faut toujours s'assurer d'être à jour sur la branche dev:

- `git status`
- `git pull`



2.2.3) Convention de nommage des branches

Les branches doivent être sous le format suivi et ne pas contenir des informations non conforme comme le prénom ou des émojis. Puisque chaque ticket est individuel, les branches sont destinées à être utilisées par un seul dev.

cf convention github de nommage de branche (on peut considérer que le type de travail et l'ID du ticket suffisent):

Exemple: feat/PLOK-1

2.3) Phase de test avant de merge

2.3.1) Respect des spécifications

Il faut s'assurer que le travail effectué répond aux attentes listées dans le détail du ticket jira.

2.3.2) Passer les tests, vérification de la feature

Au moyen de tests unitaires, s'assurer du bon fonctionnement de la feature.

2.3.3) Comment merge?

Une fois votre développement terminé sur votre branche il est maintenant temps de merge, c'est-à-dire fusionner votre travail dans la branche dev.

La question se pose de la méthode à utiliser. Nous allons utiliser le rebase avant toute ouverture de pull request afin de faciliter le travail des personnes chargées des reviews de code:

- Enregistrer localement son travail
- `git status`
- Afin de vérifier les documents que vous souhaitez commit et ne pas commit des fichiers non désirés par inadvertance. Ajoutez les documents sensibles au `.gitignore`
- `git add .`
- `git commit -m <le message doit respecter la conventional commit>` :
 - Conventional commit: Message de commit soumis à la [Conventional Commits Cheatsheet · GitHub](#), vérification assuré par [husky](#).
- Rebase
 - Le rebase prend toute son importance si des changements ont été opérés sur la branche develop depuis la création de notre branche ([reponse stackoverflow](#)). Mettre sa propre branche à jour avant d'ouvrir la pull request facilitera le travail en code review et supprimera nombre de potentiel conflit que cela pourrait engendrer. Utiliser le rebase implique que les branches sont et doivent rester individuelles afin de ne pas complexifier le debug.
- `git status` (afin de s'assurer d'une branch clean)
- `git checkout develop`
- `git pull`
- `git checkout <nom de sa branche>`
- `git rebase develop`
- `git push <nom de sa branche>`



2.3.4) Open a Pull Request

Une fois le travail push il suffit de se rendre sur la page internet du repo distant et de suivre les étapes du site pour ouvrir la pull request.

(Attention à bien sélectionner develop pour votre pull request)

Description:

La description de la pull request est une étape importante pour la documentation et l'historique mais aussi pour faciliter le travail des code reviewers.

Le ticket Jira:

Ajouter le lien vers votre ticket Jira ou c/c la description de celui-ci directement dans la description

2.3.5) Code Review

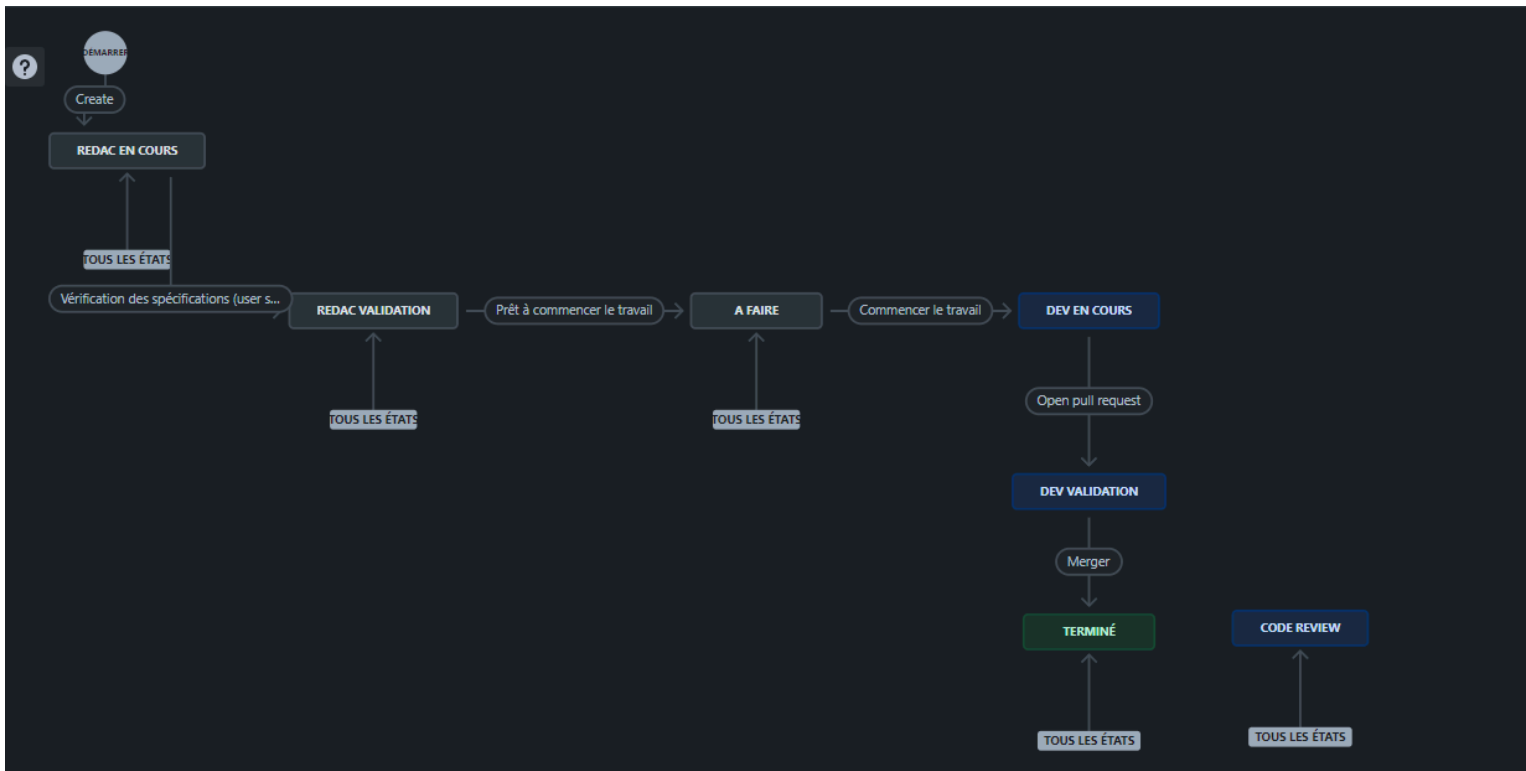
Le code review est à la charge de 2 personnes minimum avant de pouvoir valider la pull request.

2.3.6) Validation et Merge

Une fois la pull request révisée par au moins 2 personnes, on peut à présent merge la branche dans la branche develop puis supprimer la branche feature.



3) Description Workflow Jira



- **Rédac en cours** : concerne les users stories en cours de rédaction
- **Rédac validation**: concerne les users stories à valider
- **A faire**: concerne les US prêts à être commencer, dont la rédaction à été terminée et validé
 - idéalement les US tags “A faire” (dont la rédaction est terminée et validée donc), sont prêts à être placés dans le sprint en cours. Mais il n’est pas nécessaire qu’ils le soient pour être placés dans le backlog du sprint en cours.
- **Dev en cours**: concerne ce qui est en cours de développement
 - généralement déjà placé dans le sprint en cours
- **Dev validation**: concerne les US dont le développement est terminé, généralement la branche fait l’objet d’une pull request et demande la validation de code à 2 personnes de l’équipe comme décrit dans le git flow ci-dessus
- **Terminé**: concerne les US dont la pull request à été validé et merge à la branche develop